

Tezza Lending Platform — Technical Overview

ByteSurgeonAmos

September 2026

1 Executive Summary

2 System Architecture

2.1 Module Map

2.2 Infrastructure

3 Database Design

3.1 Schema Conventions

3.2 Migration History

3.3 Key Table Relationships

4 Stored Procedures

4.1 Design Rationale

4.2 Procedure Reference

4.2.1 `proc_allocate_repayment(p_loan_id UUID, p_payment_amount NUMERIC)`

4.2.2 `proc_close_loan_if_paid(p_loan_id UUID)`

4.2.3 `proc_apply_late_fee(p_loan_id UUID, p_fee_amount NUMERIC)`

4.2.4 `proc_apply_daily_fee(p_loan_id UUID, p_fee_amount NUMERIC)`

4.2.5 `proc_run_overdue_sweep(OUT p_count INTEGER)`

4.3 Invocation Pattern

5 OAuth2 Authentication

5.1 Flow

5.2 JWT Structure

5.3 Pre-registered Clients

5.4 RSA Signing Key

6 API Reference

6.1 Response Envelope

6.2 Endpoints

6.2.1 Auth — `POST /auth/clients/register`

6.2.2	Products — /api/v1/products
6.2.3	Customers — /api/v1/customers
6.2.4	Loans — /api/v1/loans
6.2.5	Repayments — /api/v1/repayments
6.2.6	Notifications
7	Event-Driven Notifications
7.1	Pipeline
7.2	Event Types
7.3	Template Rendering
8	Security
8.1	Threat Mitigations
8.2	Auth Endpoint Access
9	Postman Collection
9.1	File
9.2	Coverage
9.3	Variable Chain
9.4	Running
10	Running the Application
10.1	Local Development
10.2	Migrations
10.3	Tests
11	Appendix: Enum Values

1 Executive Summary

Tezza Lending is a **Spring Modulith** monolith implementing a full lending lifecycle for a Kenyan fintech context. It handles loan product configuration, customer onboarding, loan disbursement, installment scheduling, FIFO repayment allocation, automated sweep jobs, and event-driven notifications over Apache Kafka.

Key design decisions:

- **Spring Modulith** enforces module boundaries at build time — no cross-module `internal/` imports allowed.
- **PostgreSQL stored procedures** handle all business-critical mutations (FIFO allocation, fee accrual, status transitions) atomically.
- **OAuth2 client_credentials** flow (Spring Authorization Server) — no user sessions; designed for microservice and mobile app integration.

- **Flyway** manages all schema migrations including stored procedure DDL.
 - All 34 API endpoints are covered by a Postman collection with 22 automated assertions.
-

2 System Architecture

2.1 Module Map

```
com.tezza.lending
├─ auth/           OAuth2 Authorization Server + client registration
├─ product/        Loan products and fee configuration
├─ customer/       Customer profiles and loan limits
├─ loan/           Loan lifecycle, installment scheduling, sweep jobs
├─ repayment/      Payment processing and FIFO allocation
├─ notification/   Kafka-driven multi-channel notification dispatch
└─ shared/         ApiResponse, exceptions, cross-cutting utilities
```

Each module follows the pattern:

```
<module>/
├─ api/            Public surface (DTOs, service interfaces) – importable by
other modules
└─ internal/       Implementation details – invisible to other modules
    ├─ entity/      JPA entities
    ├─ repository/  Spring Data repositories
    └─ service/     Service implementations
```

Spring Modulith enforces this at build time via `ApplicationModules.verify()` in `ModularityTest`. Any import of an `internal/` package from outside the module fails the build.

2.2 Infrastructure

Component	Technology	Purpose
Application	Spring Boot 3.3 / Java 21	Runtime
Modularity	Spring Modulith 1.2	Boundary enforcement, event publication
Database	PostgreSQL 16	Persistence + stored procedures
Migrations	Flyway 10	Schema versioning
Messaging	Apache Kafka	Async notification pipeline
Auth	Spring Authorization Server	OAuth2 token issuance
API Docs	SpringDoc OpenAPI 3	Swagger UI at /swagger-ui.html
Containers	Docker Compose	Local dev infrastructure

3 Database Design

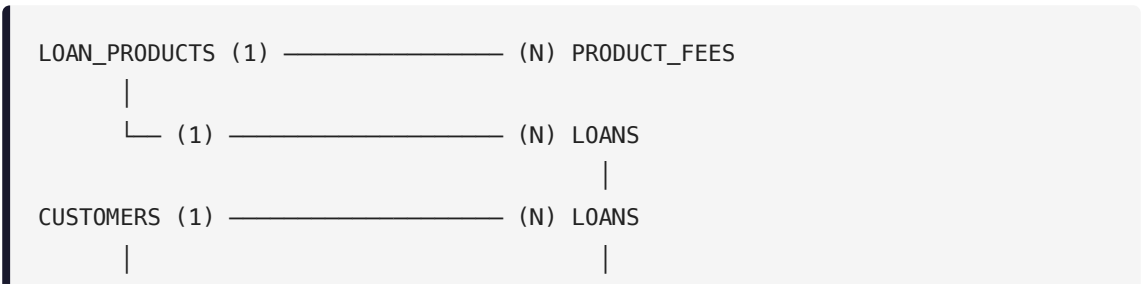
3.1 Schema Conventions

- All table and column names are **UPPERCASE**.
- Primary keys are UUID generated by `gen_random_uuid()`.
- Audit columns (`CREATED_AT`, `UPDATED_AT`, `CREATED_BY`, `UPDATED_BY`) on all core tables.
- Enum values stored as `VARCHAR` with `CHECK` constraints.

3.2 Migration History

Version	File	Contents
V1	V1__create_products.sql	LOAN_PRODUCTS, PRODUCT_FEES
V2	V2__create_customers.sql	CUSTOMERS, CUSTOMER_LOAN_LIMITS
V3	V3__create_loans.sql	LOANS, LOAN_INSTALLMENTS
V4	V4__create_repayments.sql	REPAYMENTS
V5	V5__create_notifications.sql	NOTIFICATION_TEMPLATES, NOTIFICATION_RULES, NOTIFICATION_LOGS
V6	V6__create_auth.sql	OAUTH2_REGISTERED_CLIENT (Spring Authorization Server schema)
V7	V7__create_procedures.sql	Five PostgreSQL stored procedures
V99	V99__seed_data.sql	Reference data: 2 products, 3 customers, 8 templates, 8 rules
V100	V100__create_event_publication.sql	EVENT_PUBLICATION for Spring Modulith async tracking

3.3 Key Table Relationships



└─ (1) CUSTOMER_LOAN_LIMITS └─ (N) LOAN_INSTALLMENTS
 └─ (N) REPAYMENTS

NOTIFICATION_TEMPLATES — (eventType, channel) unique
NOTIFICATION_RULES — drive dispatch per event
NOTIFICATION_LOGS — audit trail per customer
EVENT_PUBLICATION — Spring Modulith async event registry

4 Stored Procedures

4.1 Design Rationale

All business-critical mutations that involve reading then writing (FIFO allocation, balance deduction, status transitions, fee accrual) are implemented as **PostgreSQL stored procedures** rather than application-level logic. This provides:

1. **Atomicity without distributed transactions** — the entire mutation happens in one database statement within the caller's transaction.
2. **Elimination of TOCTOU race conditions** — reading installment balances and writing payments in the same PL/pgSQL block prevents concurrent repayments from double-allocating against the same installment.
3. **Bulk sweep performance** — `proc_run_overdue_sweep` updates thousands of loans in one `UPDATE ... WHERE` without OOM risk.
4. **Auditability** — stored procedure DDL lives in Flyway migrations (V7), versioned alongside schema changes.

4.2 Procedure Reference

4.2.1 `proc_allocate_repayment(p_loan_id UUID, p_payment_amount NUMERIC)`

Called by: `RepaymentServiceImpl.processRepayment()`

Allocates a payment amount across a loan's open installments in **FIFO order** (earliest due date first).

Algorithm:

```
remaining = p_payment_amount
FOR each installment WHERE loan_id = p_loan_id AND status = 'PENDING'
  ORDER BY due_date ASC
DO
  apply = MIN(remaining, installment.amount_due)
  installment.amount_due -= apply
  IF installment.amount_due = 0 THEN installment.status = 'PAID'
  remaining -= apply
  IF remaining = 0 THEN EXIT
END FOR
```

4.2.2 proc_close_loan_if_paid(p_loan_id UUID)

Called by: RepaymentServiceImpl.processRepayment() after allocation

Atomically sets `LOANS.STATUS = 'CLOSED'` and `LOANS.CLOSED_AT = NOW()` when `OUTSTANDING_BALANCE = 0`. No-op otherwise.

4.2.3 proc_apply_late_fee(p_loan_id UUID, p_fee_amount NUMERIC)

Called by: OverdueSweepJob

Adds `p_fee_amount` to `LOANS.OUTSTANDING_BALANCE` for an overdue loan. Single UPDATE statement.

4.2.4 proc_apply_daily_fee(p_loan_id UUID, p_fee_amount NUMERIC)

Called by: DailyFeeSweepJob

Identical structure to `proc_apply_late_fee` — accrues a daily fee on OPEN or OVERDUE loans.

4.2.5 proc_run_overdue_sweep(OUT p_count INTEGER)

Called by: OverdueSweepJob

```
UPDATE LOANS
SET STATUS = 'OVERDUE', UPDATED_AT = NOW()
```

```
WHERE STATUS = 'OPEN'  
AND DUE_DATE < CURRENT_DATE;  
  
GET DIAGNOSTICS p_count = ROW_COUNT;
```

Single bulk statement — transitions every past-due OPEN loan to OVERDUE and returns the count of affected rows.

4.3 Invocation Pattern

All stored procedures are called via `EntityManager.createNativeQuery()`:

```
entityManager.createNativeQuery("CALL proc_allocate_repayment(:loanId,  
    :amount)")  
    .setParameter("loanId", loan.getId())  
    .setParameter("amount", request.getAmount())  
    .executeUpdate();  
  
// Refresh entity from DB after SP mutation  
entityManager.refresh(loan);
```

`entityManager.refresh(loan)` is called after each SP invocation so the Java entity reflects the database state (balance, status) without a separate SELECT.

5 OAuth2 Authentication

5.1 Flow

Spring Authorization Server implements the `client_credentials` grant. There are no user sessions or authorization codes. Clients authenticate with their `clientId` and `clientSecret` (HTTP Basic), receive a signed JWT, and include it as a `Bearer` token on API calls.

5.2 JWT Structure

The JWT payload includes:

```
{
  "sub": "tezza-admin-client",
  "iss": "http://localhost:8080",
  "iat": 1234567890,
  "exp": 1234571490,
  "scope": "ADMIN",
  "roles": ["ROLE_ADMIN"],
  "client_id": "tezza-admin-client"
}
```

The roles claim is injected by `OAuth2TokenCustomizer<JwtEncodingContext>` and maps OAuth2 scopes to Spring Security `ROLE_*` authorities. Controllers use `@PreAuthorize("hasRole('ADMIN')")` — Spring Security prepends `ROLE_`, so the stored role must be `ROLE_ADMIN`.

5.3 Pre-registered Clients

Client ID	Secret (dev default)	Scope	Token TTL
tezza-admin-client	admin-secret	ADMIN	1 hour
tezza-mobile-client	mobile-secret	CUSTOMER	2 hours

Clients are stored in `OAUTH2_REGISTERED_CLIENT` (`JdbcRegisteredClientRepository`). At startup, if a client does not exist it is created.

5.4 RSA Signing Key

JWTs are signed with an RSA-2048 key. Set `TEZZA_JWT_PRIVATE_KEY` (base64-encoded PKCS8 DER) for key persistence across restarts. Without it, a new key is generated each restart and existing tokens become invalid.

```
# Generate a stable key for production
openssl genrsa 2048 | openssl pkcs8 -topk8 -nocrypt -outform DER | base64
```

6 API Reference

6.1 Response Envelope

All endpoints return a standard `ApiResponse<T>`:

```
{
  "status": 0,
  "message": "Product created",
  "correlationId": "550e8400-e29b-41d4-a716-446655440000",
  "data": { ... }
}
```

`status: 0` = success, `status: 1` = error. `correlationId` is a UUID per response for log tracing. `data` is omitted (`null`) on error responses.

6.2 Endpoints

6.2.1 Auth — `POST /auth/clients/register`

Register a new OAuth2 client (admin portal, microservice integration).

Body:

```
{
  "clientId": "my-service",
  "clientSecret": "min-16-char-secret",
  "clientName": "My Service",
  "scope": "CUSTOMER"
}
```

6.2.2 Products — /api/v1/products

Method	Path	Role	Notes
POST	/api/v1/products	ADMIN	Creates product; unique
GET	/api/v1/products	Any	?activeOnly=true
GET	/api/v1/products/{id}	Any	—
PUT	/api/v1/products/{id}	ADMIN	Full replacement
DELETE	/api/v1/products/{id}	ADMIN	Soft-deactivate (set active=false)
POST	/api/v1/products/{id}/fees	ADMIN	feeType: SERVICE_FEE, DAILY_FEE, LATE_FEE
DELETE	/api/v1/products/{id}/fees/{feeId}	ADMIN	Removes fee

Fee types:

Type	calculationType	daysAfterDue	Applied by
SERVICE_FEE	FIXED or PERCENTAGE	0	At disbursement
DAILY_FEE	FIXED or PERCENTAGE	0	DailyFeeSweepJob (daily)
LATE_FEE	FIXED or PERCENTAGE	≥1	OverdueSweepJob

6.2.3 Customers — /api/v1/customers

All endpoints require ADMIN scope.

Method	Path	Notes
POST	/api/v1/customers	Email + nationalId must be unique
GET	/api/v1/customers/{id}	—
PUT	/api/v1/customers/{id}	—
GET	/api/v1/customers/{id}/loan-limit	Returns 404 until limit is set
PUT	/api/v1/customers/{id}/loan-limit	Upserts the limit record

6.2.4 Loans — /api/v1/loans

Disburse a loan (POST):

```
{
  "customerId": "uuid",
  "productId": "uuid",
  "amount": 10000.00,
  "loanType": "INSTALLMENT",
  "billingCycleType": "INDIVIDUAL",
  "numInstallments": 3
}
```

loanType: LUMP_SUM | INSTALLMENT billingCycleType: INDIVIDUAL (one installment per billing period) | CONSOLIDATED (single due date)

On disburse: installments are generated, service fee is calculated and applied, a LOAN_CREATED event is published.

6.2.5 Repayments — /api/v1/repayments

Process repayment (POST):

```
{
  "loanId": "uuid",
  "amount": 3500.00,
  "reference": "MPESA-ABC123",
}
```

```
"channel": "MPESA"  
}
```

channel: MPESA | BANK | CASH

Processing sequence: 1. Validate loan is OPEN or OVERDUE. 2. Deduct amount from `LOANS.OUTSTANDING_BALANCE`. 3. CALL `proc_allocate_repayment(loanId, amount)` — FIFO installment allocation. 4. CALL `proc_close_loan_if_paid(loanId)` — auto-close if balance = 0. 5. Persist `REPAYMENTS` record. 6. Refresh entity + publish `REPAYMENT_ACK` event.

6.2.6 Notifications

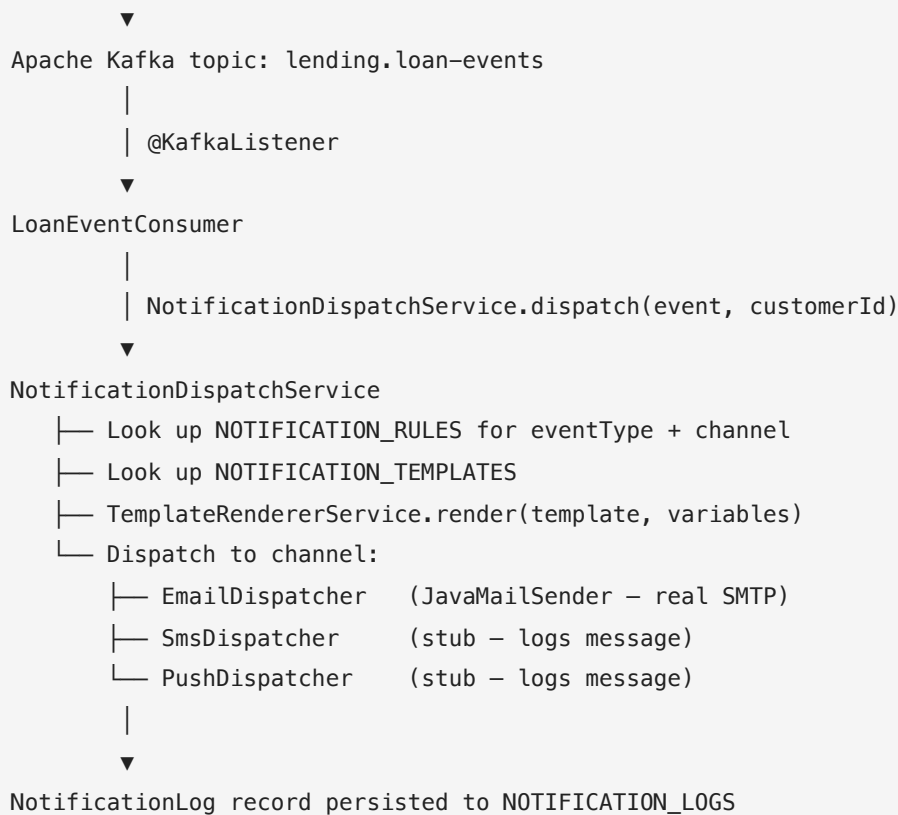
Template variables available in `bodyTemplate`:

Variable	Source
<code>{{customerName}}</code>	Customer first + last name
<code>{{loanNumber}}</code>	Loan number (TZ-YYYY-XXXXXXXX)
<code>{{amount}}</code>	Payment or loan amount
<code>{{dueDate}}</code>	Installment or loan due date
<code>{{balance}}</code>	Outstanding balance after payment

7 Event-Driven Notifications

7.1 Pipeline

```
Loan/Repayment Service  
|  
| ApplicationEventPublisher.publishEvent(LoanEvent)  
▼  
LoanEventListener (@EventListener @Async)  
|  
| KafkaTemplate.send("lending.loan-events", LoanEventMessage)
```



7.2 Event Types

Event	Published when
LOAN_CREATED	Loan is disbursed
DUE_REMINDER	DueDateReminderJob fires (3 days before due)
REPAYMENT_ACK	Repayment is processed
OVERDUE_NOTICE	OverdueSweepJob marks loan OVERDUE

7.3 Template Rendering

Templates use `{{variable}}` syntax. `TemplateRenderService` substitutes all `{{key}}` tokens from a provided context map using a regex replace:

```
Pattern.compile("\\{\\{([\\w+])\\}\\}")
```

8 Security

8.1 Threat Mitigations

Threat	Mitigation
Secret exposure	All secrets externalized to env vars; no hardcoded values in committed code
Role privilege escalation	<code>hasRole('ADMIN')</code> guards write endpoints; JWT roles claim maps scopes → <code>ROLE_*</code> authorities
Double-prefix auth bug	<code>hasRole('ADMIN')</code> (not <code>hasRole('ROLE_ADMIN')</code>) — Spring Security auto-prepends <code>ROLE_</code>
Enumeration via error messages	Client conflict returns "Client registration failed" (not the conflicting clientId)
Token invalidation on restart	Stable RSA key via <code>TEZZA_JWT_PRIVATE_KEY</code> env var; ephemeral key logs a warning
Actuator exposure	Only <code>/actuator/health</code> exposed, <code>show-details: never</code>
SQL injection	All queries via JPA / named parameters; no string concatenation in queries

8.2 Auth Endpoint Access

Endpoint	Public	CUSTOMER	ADMIN
POST <code>/oauth2/token</code>	✓	—	—

Endpoint	Public	CUSTOMER	ADMIN
POST /auth/clients/register	✓	—	—
GET /api/v1/products	—	✓	✓
POST /api/v1/products	—	—	✓
POST /api/v1/loans	—	✓	✓
GET /api/v1/loans	—	—	✓
POST /api/v1/repayments	—	✓	✓
All /api/v1/customers/**	—	—	✓
All /api/v1/notifications/**	—	—	✓
GET /actuator/health	✓	—	—

9 Postman Collection

9.1 File

tezza-lending.postman_collection.json — included in the repository root.

9.2 Coverage

Section	Requests	Assertions
Auth	3	4
Products	7	3
Customers	5	2
Loans	7	3

Section	Requests	Assertions
Repayments	6	3
Notifications	5	4
Health	1	2
Total	34	22

Result: 34 requests, 22 assertions, 0 failures — verified against a live instance backed by PostgreSQL 16 and Kafka.

9.3 Variable Chain

The collection uses Postman collection variables. Each create request captures the returned ID and stores it for subsequent requests:

```
adminToken ← Get Admin Token
productId  ← Create Product
feeId      ← Add Service Fee to Product
customerId ← Create Customer
loanId     ← Disburse Loan (Installment) / Repayment test loan
```

{{timestamp}} is used in product names, customer emails, and national IDs to ensure uniqueness across multiple collection runs.

9.4 Running

```
# Newman (Postman CLI)
npm install -g newman
newman run tezza-lending.postman_collection.json --delay-request 300

# Or import into Postman Desktop:
# File → Import → tezza-lending.postman_collection.json
```

10 Running the Application

10.1 Local Development

```
# Infrastructure
docker compose up -d

# Application
mvn spring-boot:run

# Swagger UI
open http://localhost:8080/swagger-ui.html
```

10.2 Migrations

Flyway runs automatically on startup. All 9 migration versions (V1–V7, V99, V100) apply in order on a fresh database. On an existing database, only unapplied migrations run.

To inspect migration status:

```
SELECT version, description, installed_on, success
FROM flyway_schema_history
ORDER BY installed_rank;
```

10.3 Tests

```
# Full suite (Testcontainers spins up PostgreSQL automatically)
mvn test

# Coverage
mvn verify && open target/site/jacoco/index.html
```

11 Appendix: Enum Values

Enum	Values
TenureType	DAYS, MONTHS
FeeType	SERVICE_FEE, DAILY_FEE, LATE_FEE
CalculationType	FIXED, PERCENTAGE
LoanType	LUMP_SUM, INSTALLMENT
BillingCycleType	INDIVIDUAL, CONSOLIDATED
LoanStatus	OPEN, OVERDUE, CLOSED, WRITTEN_OFF, CANCELLED
InstallmentStatus	PENDING, PAID, OVERDUE
PaymentChannel	MPESA, BANK, CASH
EventType	LOAN_CREATED, DUE_REMINDER, REPAYMENT_ACK, OVERDUE_NOTICE
NotificationChannel	EMAIL, SMS, PUSH
NotificationStatus	SENT, FAILED, PENDING